ML Experiment Number 10

Student Name : Chinmayi Milind Chavan

Batch :CE1

Roll Number :CE03

Aim

To implement DBSCAN clustering using Python for customer segmentation on the Mall Customer Segmentation dataset and identify arbitrary-shaped clusters along with spatial outliers (noise).

Objectives

1. To understand the principles of Density-Based Spatial Clustering of Applications with Noise (DBSCAN).
2. To implement DBSCAN clustering using Python.3. To preprocess and feature-scale tabular customer data using StandardScaler.
3. To determine optimal hyperparameters (ε and $min_samples$) using k-distance graph analysis.
4. To identify core points, border points, and spatial outliers (noise points denoted by label -1).
5. To evaluate clustering performance using metrics like Silhouette Score.

Double-click (or enter) to edit

Relevant Theory

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is an unsupervised non-parametric clustering algorithm that groups data points based on local density estimations. Unlike centroid-based algorithms (e.g., K-Means), DBSCAN can discover arbitrary-shaped clusters and automatically isolate noisy data points without requiring the user to specify the total number of clusters in advance.

Core Concepts and Parameters:

1. ε (Epsilon): The maximum radius of the neighborhood around a given data point x.
2. $min_samples$ (MinPts): The minimum number of points within the ε-neighborhood required to form a dense region.

Point ClassificationsCore Point:

- A point that has at least $min_samples$ within its ε-neighborhood.
- Border Point: A point that is within the ε-neighborhood of a Core Point but has fewer than $min_samples$ in its own neighborhood.
- Noise Point (Outlier): A point that is neither a Core Point nor a Border Point. Noise points are assigned a cluster label of -1.

Dataset Information

Get the Mall Customer Segmentation dataset from Kaggle using the following link: https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

The dataset contains spending patterns and demographic attributes of mall customers:

- CustomerID: Unique identifier for each customer.
- Gender: Male or Female.
- Age: Age of the customer.
- Annual Income (k$): Annual income of the customer in thousands of dollars.
- Spending Score (1-100): Score assigned based on customer behavior and purchasing patterns.

For clustering, we will use: Annual Income (k$) and Spending Score (1-100).

Task 1: Import Libraries and Load Dataset

- Load essential libraries (numpy, pandas, matplotlib, seaborn, sklearn).
- Load the Mall Customer Segmentation dataset (Mall_Customers.csv).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
import urllib.request

# Download the Mall Customers dataset directly
url = 'https://raw.githubusercontent.com/SteffiPeTaffy/machineLearningAZ/master/Machine%2
urllib.request.urlretrieve(url, 'Mall_Customers.csv')

# Load dataset
df = pd.read_csv('Mall_Customers.csv')
print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset shape: (200, 5)
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Double-click (or enter) to edit

Task 2: Feature Selection and Standardization

- Select target features (Annual Income (k) and Spending Score (1-100)).
- Scale features using Standard Scaler to bring feature dimensions to a comparable range ($\mu = 0, \sigma = 1$), which is crucial for distance-based neighborhood computations.

```
# Task 2: Feature Selection and Standardization
# Selecting Annual Income (k$) and Spending Score (1-100) for clustering
features = ['Annual Income (k$)', 'Spending Score (1-100)']
X = df[features].values

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Scaled features sample:")
print(X_scaled[:5])
```

```
Scaled features sample:
[[-1.73899919 -0.43480148]
 [-1.73899919  1.19570407]
 [-1.70082976 -1.71591298]
 [-1.70082976  1.04041783]
 [-1.66266033 -0.39597992]]
```

Task 3: Hyperparameter Selection via k-Distance Graph

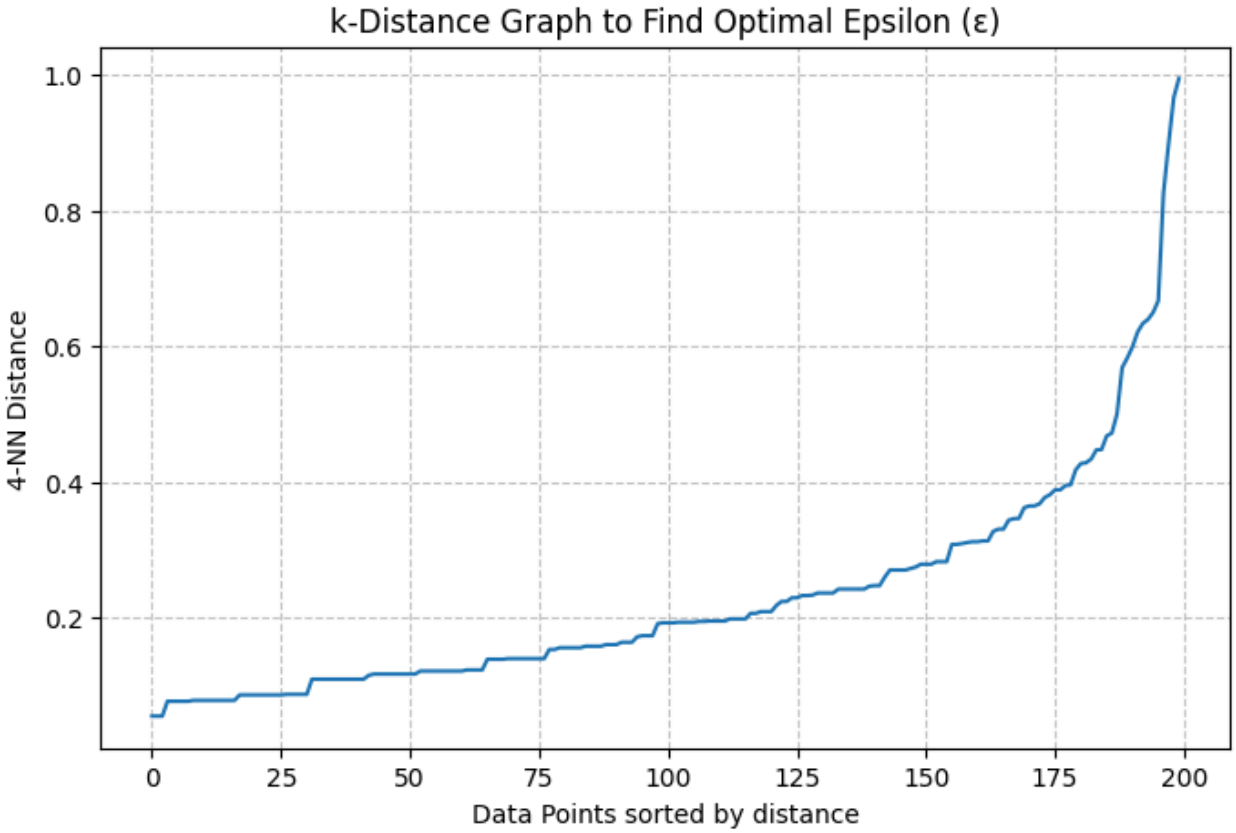
- Compute Nearest Neighbors using NearestNeighbors ($k = min_samples$).
- Plot sorted k-distances to locate the "elbow point," which indicates the optimal value for ε (Epsilon).

```
# Task 3: Hyperparameter Selection via k-Distance Graph
# Typically, min_samples is chosen as 2 * dimensions = 4.
min_samples = 4

# Compute nearest neighbors
neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)

# Sort the distances of the k-th nearest neighbor (column index min_samples-1)
sorted_distances = np.sort(distances[:, min_samples - 1], axis=0)

# Plot the k-distance graph to identify the 'elbow'
plt.figure(figsize=(8, 5))
plt.plot(sorted_distances)
plt.title('k-Distance Graph to Find Optimal Epsilon (ε)')
plt.xlabel('Data Points sorted by distance')
plt.ylabel(f'{min_samples}-NN Distance')
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



Task 4: Implement DBSCAN Clustering

- Instantiate sklearn.cluster.DBSCAN with the identified ε and $min_samples$.
- Fit the model on scaled data to generate cluster assignments and noise labels (-1).

```
# Task 4: Implement DBSCAN Clustering
# Based on standard scaling and 4-NN distance, an epsilon around 0.3 to 0.4 is generally
epsilon = 0.35
min_samples = 4

dbscan = DBSCAN(eps=epsilon, min_samples=min_samples)
dbscan.fit(X_scaled)

# Get cluster assignments
labels = dbscan.labels_
print("Unique labels generated by DBSCAN:", np.unique(labels))
```

```
Unique labels generated by DBSCAN: [-1  0  1  2  3  4  5  6]
```

Task 5: Evaluate DBSCAN Clustering

- Cluster Count: Calculate the number of clusters generated (excluding noise points).
- Noise Point Analysis: Count and evaluate identified spatial outliers (label -1).
- Silhouette Score: Calculate Silhouette Score to evaluate cluster cohesion and separation.

```
# Task 5: Evaluate DBSCAN Clustering
# Number of clusters (excluding noise points if present)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
```

```
n_noise = list(labels).count(-1)

print(f"Estimated number of clusters: {n_clusters}")
print(f"Estimated number of noise points: {n_noise}")

# Calculate Silhouette Score only if we have more than 1 cluster (excluding noise)
if n_clusters > 1:
    score = silhouette_score(X_scaled, labels)
    print(f"Silhouette Coefficient: {score:.3f}")
else:
    print("Silhouette Score cannot be computed (less than 2 clusters found).")
```

```
Estimated number of clusters: 7
Estimated number of noise points: 16
Silhouette Coefficient: 0.426
```

Task 6: Analyze Results and Cluster Statistics

- Extract cluster labels and append them back to the original dataset.
- Calculate mean statistics (e.g., average Annual Income and Spending Score) per cluster to interpret final customer segments and evaluate detected noise points.

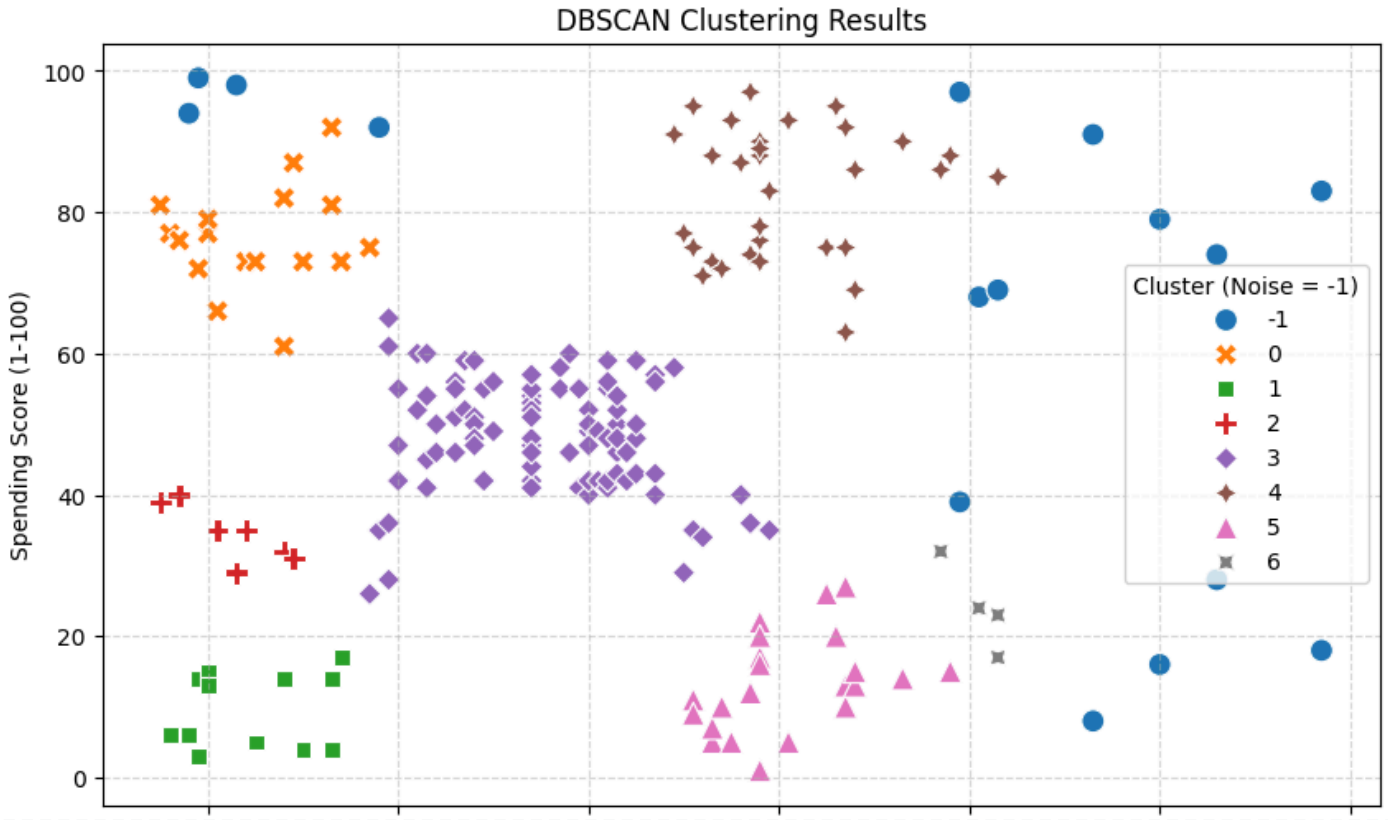
```
# Task 6: Analyze Results and Cluster Statistics
# Assign labels back to the original DataFrame
df['Cluster'] = labels

# Calculate mean statistics for each cluster
cluster_stats = df.groupby('Cluster')[['Age', 'Annual Income (k$)', 'Spending Score (1-10
print("Mean statistics per cluster:")
display(cluster_stats)

# Plot the resulting clusters
plt.figure(figsize=(10, 6))
sns.scatterplot(
    x=df['Annual Income (k$)'],
    y=df['Spending Score (1-100)'],
    hue=df['Cluster'],
    palette='tab10',
    style=df['Cluster'],
    s=100
)
plt.title('DBSCAN Clustering Results')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend(title='Cluster (Noise = -1)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

Mean statistics per cluster:

Cluster	Age	Annual Income (k$)	Spending Score (1-100)
-1	33.750000	93.250000	65.812500
0	24.411765	25.235294	76.352941
1	48.750000	24.583333	9.583333
2	36.714286	22.428571	34.428571
3	42.797753	55.022472	48.325843
4	32.812500	80.500000	82.562500
5	41.217391	80.956522	12.782609
6	41.500000	101.000000	24.000000



Next steps: Generate code with `cluster_stats` New interactive sheet

Conclusion

The experiment successfully implemented DBSCAN clustering on the Mall Customer Segmentation dataset. Feature standardization ensured balanced neighborhood distance calculations across income and spending metrics. By analyzing k-distance graphs, optimal ε and $min_samples$ parameters were selected, enabling the discovery of density-defined customer segments and explicit identification of spatial noise points (outliers) without requiring a pre-defined cluster count.